Back to the case

Situations · Reading version

THEME 1 / STATE OF WORK AND PUBLICATION

A repository for research – one publication for another.

A suitable research repository is not yet a data publication.

Elif's question

Lea gives Elif access to her research repo. Elif wants to investigate rhyme forms and asks: "Which TEI folder should I take?

Reasonable for Lea's investigation

The repo preserves the acquired sources, the standardized data and the recorded entries of individual studies. The same book is thus available in several well-founded versions. This structure documents Lea's research path. Elif, on the other hand, needs a clearly identified corpus stock for a new study.

REQUESTS

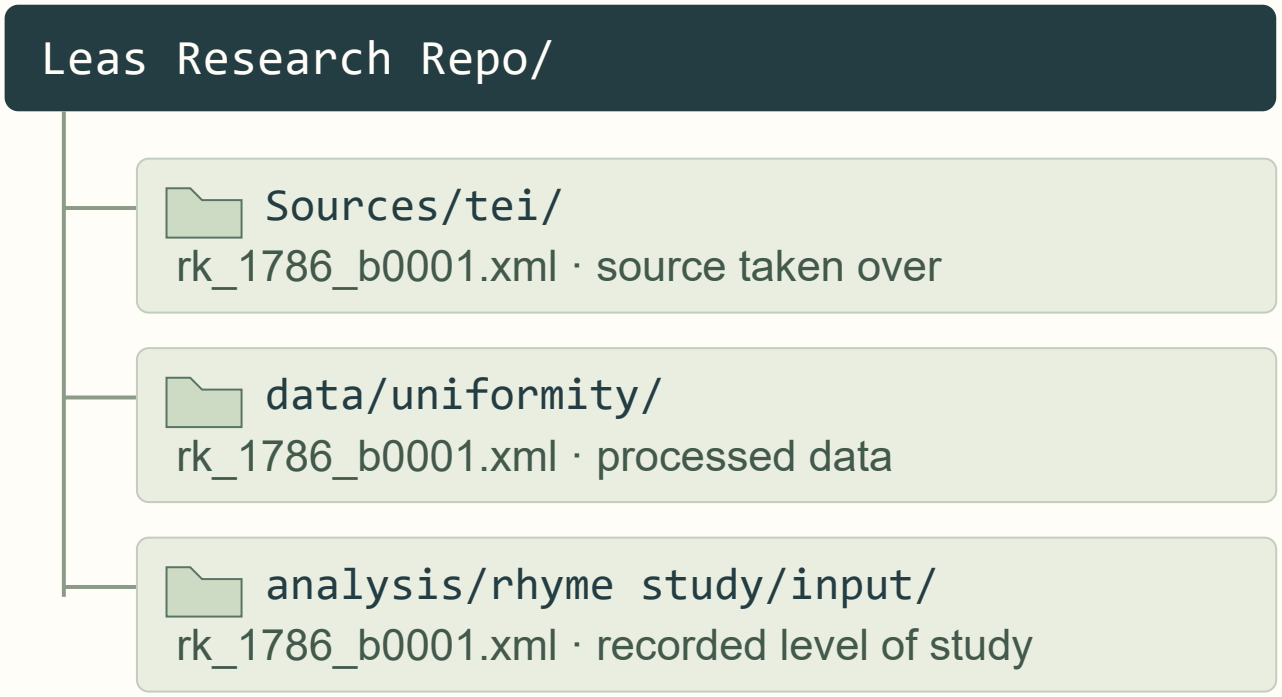
A publication repository is not simply a research repository made public!

A good research repository explains how a study was developed. The data publication explains which inventory others should use for new studies. We set up our own curated corpusrepo for this purpose.

▼ Example and Tip

Example: Research repo: sources/tei/rk_1786_b0001.xml → data/uniformized/ZXq1 → analyses/rhyme study/entry/ZX Q2. Publication repo: exactly one relevant corpus under data/tei / rk_1786_b0001.xml.

Tip: Note publication concept before the first approval in Git.



A research path: Source → Preparation → State of study.

▼ Read graphic as text

Lea's research filing

1. Leas Research Repo/
2. Sources/tei/
3. rk_1786_b0001.xml · source taken over
4. data/uniformity/
5. rk_1786_b0001.xml · processed data
6. analysis/rhyme study/input/
7. rk_1786_b0001.xml · recorded level of study
8. A research path: Source → Preparation → State of study.

For the own study, the separate processing positions are useful.

TWO PURPOSES · TWO OFFERS

Useful for the study – differently prepared for publication

In the research repo	Why make sense there?	In the publication repository
Sources/tei/rk_1786_b0001.xml	Approved version for comparison with the preparation.	Source and origin in metadata; no competing corpus folder.
data/uniformity/rk_1786_b0001.xml	Prepared version for Leas evaluations.	After verification of the relevant collection under **data/tei/rk_1786_b0001.xml**.
analysis/rhyme study/input/rk_1786_b0001.xml	Fixed input, with which a study result emerged.	The study calls release and selection. Your frozen section is not offered as a second corpus version.

The research repository is appropriate for its purpose. Clear entry, applicable rules and a released inventory are added to the publication.

View the full recommended publication structure ↑

That's how we'd do it.

You can do that.

1. Note publication concept before the first approval in Git.
2. Apply audited book files from data/uniformized to data/tei of the publication repositorys.
3. README, add rules and entry; let the first release be tested externally.

Concrete example

Research repo: sources/tei/rk_1786_b0001.xml → data/uniformized/ZXq1 → analyses/rhyme study/entry/ZX Q2. Publication repo: exactly one relevant corpus under data/tei / rk_1786_b0001.xml.

Done if: A new user recognizes without question which files and which version to examine.

Git helps: Git keeps publication concept, data and corrections together in a comprehensible way. GitHub makes README, releases, issues and audited contributions accessible. The

team decides which work history becomes public.

▼ For workshop and self-study: a question

What would anyone know who will start researching with our repo today without us?

TOPIC 2 / QUALITY TEST

The script's running.

A content can disappear, although the previous evaluation continues.

An unnoticed omission

Lea's new preparation takes over the puzzle texts, but loses the solution part: It was captured in a separate XML element in the previous TEI structure and is no longer included in the newly generated TEI data. Python-Evaluation only counts puzzles and verses: The numbers remain the same. Only when reading a seminar task the answer is missing.

Limiting the error

Lea and Jonas use Git to compare the versions before and after conversion. They discover that the modified converter omits the XML element containing the solutions. They correct the procedure and create a small new regression test that reproduces this specific error: a riddle with a separately encoded solution must pass through the conversion intact. The test then checks that the solution and its link to the riddle have been preserved.

REQUESTS

A successful analysis proves no complete data.

We check structure, relationships and content retention separately. Automatic tests are complemented by a targeted comparison of sources.

▼ Example and Tip

Example: Two puzzles and two solutions before processing; two puzzles and no solution afterwards. XML and Demo-schema exist, the inventory comparison reports the loss.

Tip: Before and after a transformation, compare all elements and content that should be retained, as well as IDs and references.

THE OWN SKRIPT DOESN'T USE SOLUTIONS

Two solutions are missing. The puzzle number remains the same.

The new processing takes over the main part, but leaves the solution part away. The sample test performed shows:

Check	Before	After
XML and Demo-schema	collection	collection
Counted puzzles	2	2
Solutions received	2	0
Expected collection?	Yes	**Yes**

Two puzzles and two solutions. The known collection is complete.

```
<!-- In comparison, a complete part disappears: -->
- <back>
-   <!-- Two solutions with assignment to their puzzles -->
- </back>
```

What complements the test? Compare inventory and references, investigate unexpected losses and read vulnerable sites against the source. Also valid files can be technically incorrect or incomplete.

Display of the actually executed artificial example. The selection does not perform a XML check in the browser. Complete files and diff are in the sample package.

▼ A second mistake: solutions counted as puzzles

EXPERIENCE · ARTIFICIAL EXAMPLES

What's green?

The template contains two puzzles and two separate solutions. The original schema allows collections without solutions; it does not know this source.

4 entries counted as puzzles, 0 solutions. schema and existing references: passed. Known source features: not met.

Executed test report of artificial examples

Check	Incorrect import	Corrected import
Demo schema	collection	collection
IDs and existing references	collection	collection
Puzzles / Solutions	4 / 0	2 / 2
Known source structure	Not met	Fulfilled

▼ View the assignment in TEI

```
<div type="riddle" xml:id="riddle-demo-001">
  <p>Ich habe Zähne und esse nicht. Was bin ich?</p>
</div>
<!-- Solution in the separate solution part -->
<item corresp="#riddle-demo-001">Der Kamm.</item>
```

The small Relax-NG-schema and the reference checks were actually executed. This display reads the stored test report; it does not perform any new XML check in the browser. A complete corpus model needs its own documented TEI adjustment.

XML well formed?

▼ Example and Tip

Example: An unclosed element prevents the reading of a XML file.

Tip: Automatically check XML well-formedness. After that, further tests can work reliably.

→

schema respected?

▼ Example and Tip

Example: The schema checks whether elements may occur at the intended location. A missing optional solution part can still be allowed.

Tip: Version the schema with the documented corpus model; check technical completeness additionally.

→
collection and references received?

▼ Example and Tip

Example: After the export there are still two puzzles, but instead of two no solutions.

Tip: Compare expected units, identifiers and reference targets; examine each unexpected decrease.

→
Derivatives checked?

▼ Example and Tip

Example: A TXT export with solutions has more words than just an export of puzzles.

Tip: Test expected components and exclusions of extraction on a small sample.

→
Places read against source?

▼ Example and Tip

Example: The source separates puzzles and solutions, the conversion must get this relationship.

Tip: In case of changes to the conversion just read the affected source structures exactly.

→
approval documented?

▼ Example and Tip

Example: A pull request holds change, passed exams and technical approval.

Tip: Only record released changes in the next release; name known limits in the release notes.

That's how we'd do it.

You can do that.

1. Before and after a transformation, compare all elements and content that should be retained, as well as IDs and references.
2. Check the diff and affected source points in the pull request; clear unexpected losses.
3. Keep the error as a regression test and publish only after technical approval.

Concrete example

Two puzzles and two solutions before processing; two puzzles and no solution afterwards. XML and Demo-schema exist, the inventory comparison reports the loss.

Done if: The test run detects the known loss; the source control and approval are documented.

Git helps: Version comparison and designated tool stands show when content was lost and which imports are affected. A regression test accompanies the correction. Git preserves changes; the team must check its technical sense.

▼ For workshop and self-study: a question

What data could our analysis of today lose without its result changing noticeably?

Theme 3 / VERSIONS AND RESEARCH ANALYSIS

Yesterday the result was different.

Growing data need comprehensible versions.



Lea and Elif compare two different version folders.Fictional scene · AI-generated illustration

A new graphic

For her publication, Elif calculated the proportion of verse in the corpus and produced a bar chart that seemed particularly revealing. A little later, she wants to use the same script to reproduce the chart in more striking colours. But the bars now tell an unexpected story: Lea has added further collections in the meantime. Elif's script always loads the current repository version.

Repeat the used version

Using the archived earlier release and the same script, Elif can reproduce her original chart. She needs to identify the data version so that others can check her figures and conclusions. The expanded collection does not change her conclusions, although some figures are now less clear-cut. Elif records the data version, selection of evidence and procedure together. Lea can continue adding books, while earlier studies can still refer back to their specified data version.

REQUESTS

An investigation needs a certain level of data.

We publish audited releases and receive them independently of the Git host. Researchers also call selection, extraction and tooling.

▼ Example and Tip

Example: Demo-v1.0: 2 out of 4 puzzles are verses. Demo-v1.1: 8 out of 10. If the same four old documents are selected, their share remains at 50 percent.

Tip: After the approval create a protected tag and release with inventory and change notes.

STANDARD AND SELECTION STRENGTHEN

What changes the distribution?

The two old book files are byte identical in both versions. The new version complements a book with six puzzles.

corpus versionSelection
2 out of 4 puzzles are encoded as verses: 50%. The complete inventory of this release is compared.

Puzzles

Prosar puzzles

Artificial collection, no historical research results

Selection	Verse	Total	Share
Demo-v1.0	2	4	50 %
Demo-v1.1	8	10	80 %
Demo-v1.1 · old documents	2	4	50 %

▼ Same data – other export?

From the same corrected TEI file, two XSLT selections deliver 20 or 24 words separated by empty space. The second selection includes solutions. This is a technical demonstration finding, not historical language statistics.

Therefore, extraction rule, selection and tooling are part of the research basis.


```
{
  "id": "P03-DEMO-ANALYSE-01",
  "status": "künstliche Demonstration; keine archivierte historische
Datenpublikation",
  "corpus": {
    "release": "demo-v1.0",
    "path": "versionsvergleich/v1.0/data",
    "version_doi": null,
    "git_commit": null
  },
  "selection": {
    "book_ids": [
      "book-a",
      "book-b"
    ],
    "rule": "alle Rätsel der beiden genannten Beispielsammlungen"
  },
  "extraction": {
    "file": "xslt/raetseltexte-v1.xsl",
    "version": "demo-1",
    "includes_solutions": false,
    "includes_prefaces": false
  },
  "analysis": {
    "file": "run_demo.py",
    "measure": "Versrätsel / alle Rätsel",
    "definition": "riddle-div mit direktem lg = Vers; p = Prosa",
    "parameters": {
      "rounding": "keine Rundung der Bestandszahlen"
    }
  },
  "production_fields": "Bei einer tatsächlichen Forschungsanalyse werden
konkrete Version-DOI, erhaltene Commitreferenz und archivierte
Werkzeugstände ergänzt. Null ist hier ausdrücklich kein gültiges fertiges
```

```
Forschungszitat."
}
```

For this demonstration DOI and Git-Commit are expressly empty. It is not a finished research quote and no real data publication.

Commit

Exact technical revision. Can be an untested interim version.

▼ Example and Tip

Example: A commit identifies the change to a book file even if it is not yet released.

Tip: Commit messages explain the reason and scope; call the scientific data a release.

→

Tag and project release

Designated version with composition and modification instructions audited according to project rules.

▼ Example and Tip

Example: v1.1 names a tested version and supplements the list of newly added collections.

Tip: Protect published tags and do not move them tacitly to another version.

→

archived version

Received data package with metadata and version identifier. For research call the specific version used.

▼ Example and Tip

Example: A version reference leads to the received data package with which the investigation was carried out.

Tip: For the citation call the actual version used.

That's how we'd do it.

You can do that.

1. After the approval create a protected tag and release with inventory and change notes.
2. Archive data package including metadata and include the concrete version proof in the citation note.
3. For each analysis record release, selection, export rule and script version.

Concrete example

Demo-v1.0: 2 out of 4 puzzles are verses. Demo-v1.1: 8 out of 10. If the same four old documents are selected, their share remains at 50 percent.

Done if: Another person may repeat the examination from the designated data and procedures.

Git helps: Fixed data and tool stands make the comparison repeatable. Git documents inventory changes; the team must explain their scientific significance.

▼ For workshop and self-study: a question

Could someone put the same research basis back together with our information alone?

THEME 4 / SUPPLEMENTATION AND ANNOTATION

A rhyme helps the model to jump.

Additional annotation and model experiments remain assigned to their basis.



Students and Lea compare model answers and printed riddle verses.Fictional scene · AI-generated illustration

A question from the seminar

Students ask a local language model to solve rhyming two-line riddles. In one group of these riddles, the solution is also the missing rhyme word in the second line. “Does it understand the riddle, or is it simply completing the rhyme?” To investigate this, the group compares the verse with a separately stored prose adaptation that removes the rhyme clue.

Out of use, follow-up research

Elif adds annotations of form and words to the data used for this study. In particular, she identifies the riddles whose solution is the missing rhyme word. At a later workshop, the prepared data are used to compare different language models. This creates a separate small benchmark corpus: which model can solve verbal riddles in historical varieties of German? Another experiment fine-tunes a language model using the reference data; it achieves substantially better results on the separately held test data. The team decides to prepare the entire existing corpus for such tasks. The data receive an additional annotation layer. TEI allows this layer to be encoded separately and linked to the base data without changing their texts. The extended edition receives a new version number; the base release, annotations and model experiments remain traceably connected.

REQUESTS

New research should be able to build on the corpus.

Our starting point is a separately versioned additional annotation. Originals, processing, automatic layers and model experiments remain clearly distinguishable.

▼ Example and Tip

Example: Base: corpus v1.0. Addition: word annotation 1.0 with reference to its tokenization. Model test: puzzle texts in testadditions.jsonl, reference answers separated in loesungen.jsonll.

Tip: Describe annotation with base release, anchoring or word and generation methods.

SUPPLEMENTARY ANNOPTATION PUBLISHING

Our recommendation: a separate annotation layer

Own annotation layer

Our starting point for additional automatic annotation: own version, concrete base release, clear anchors and documented test status.

▼ Example and Tip

Example: A word annotation calls corpus v1.0 and its tokenization.

Tip: This is our starting point for automatic additional annotation. Keep procedures and test status visible.

Example: The word annotation 1.0 calls corpus v1.0 as the basis and documents its tokenization. The TEI base remains usable separately.

▼ Deepening: When will another path be considered?

In the main TEI

Directly in the base TEI, when the editors expand the model professionally. The change is checked and published with a new base release.

▼ Example and Tip

Example: The editorial board decides on an additional element in the basic model.

Tip: Version the model change, affected files and checks together and create a new base release.

Derived research corpus

For an independent new research edition: own repo with origin, selection and proof of transformation. This deepening needs its own care.

▼ Example and Tip

Example: A follow-up project chooses other units and maintains them in its own repo.

Tip: Prove origin and transformation; name responsibility for the new publication.

▼ Try: Rhyme and model test

A REIM AS NOTICE

What makes the riddle solveable?

> I order hair, not mud;
> my name is missing: it's the [...].

Artificial example · missing rhyme as a solution

Try: Do we give the model the rhymed verses – or an edited version without this rhyme?

Test input

Two verses with rhyme on "Schlamm" .

▼ View Separate Reference Solution

The solution is in the evaluation file, not in the test input.

An experiment needs a comprehensible structure

1. Label corpus release and selection; separate traditional texts and editing.
2. Export test entries without solutions; keep reference solutions separate.
3. Check training/test references and similar puzzles. Different IDs do not guarantee independence yet.
4. Document the model, parameters and evaluation rule before comparison.

A model can also react with plausible other answers. This must take into account the evaluation. No model is executed here and no performance value is claimed.

Small local package: data and export script ↓

That's how we'd do it.

You can do that.

1. Describe annotation with base release, anchoring or word and generation methods.
2. Specify automatic origin and test status; offer basic data without additional layer.
3. In model tests separate test inputs and solutions; test training/test selection and related evidence.

Concrete example

Base: corpus v1.0. Addition: word annotation 1.0 with reference to its tokenization. Model test: puzzle texts in testadditions.jsonl, reference answers separated in loesungen.jsonll.

Done if: The annotation is assigned to its base and can be omitted; the experiment is documented.

Git helps: Basic release, annotation and test scripts can be combined via specified versions. Issues and pull requests enable queries and verified contributions; the versioning makes the investigation comprehensible.

▼ For workshop and self-study: a question

What new research question is being made more difficult by our current markup?

THEME 5 / LOCAL USE AND TRANSFER

Does it work without Lea's computer?

A fresh download shows the quiet conditions.



Jonas works with a guide, while Lea lets him try out himself. Fictional scene · AI-generated illustration

An attempt without acclamation

Jonas loads the release to another computer and starts with README. The example still awaits Lea's network drive. Nobody in the team had noticed that: everyone worked with the same infrastructure.

Enhancing entry

Lea replaces personal paths with relative paths and documents calls, dependencies and expected results. Jonas starts again from the fresh package. This time the first examination succeeds without help. Before the end of Lea's job he also tests the instructions for the next release.

REQUESTS

The entry must work without the project environment.

We provide a small full access and the entire designated data set. Another computer is our practical countercheck.

▼ Example and Tip

Example: Instead of a path to Lea's network drive, the script korpus.xml searches relative to its own folder. Entry runs with Python default library without API or account.

Tip: Use filenames without system specific special characters and relative paths.

FROM FRESH DOWNLOAD TO THE OWN

An entry without Lea's computer

From download to first investigation

1. Unpack the small package completely.
2. With Python 3.10 or newer `python untersuchen.py` Run in the package folder.
3. Expected result: three puzzles, three solutions and three demonstrated shapes.
4. In `ausgabe/` Test entries, reference solutions and proof of results are available separately.

Only Python default library, no account and no API. The script uses relative paths. The TEI file can also be processed with other XML tools. This entry checks identifiers and solution references, not TEI-schema.

Download Usage Entry ↓ · Instructions as Markdown ↓

What is additionally checked?

- Is the relevant collection recognisable, are annotations and derivatives explained?
- Do filenames, relative paths and documented dependencies work in a different environment?
- Are data release, selection and procedures comprehensible in the result?

- Can a representative create a local sample release using the versioned care instructions?

A successful entry into use does not yet create unlimited care capacity. Contact and promised range of care remain visible.

That's how we'd do it.

You can do that.

1. Use filenames without system specific special characters and relative paths.
2. Document a call with dependencies and expected output in README.
3. Another person can have the newly unpacked package carried out; add contact and care commitment.

Concrete example

Instead of a path to Lea's network drive, the script korpus.xml searches relative to its own folder. Entry runs with Python default library without API or account.

Done if: The documented result is created in a fresh environment; missing knowledge is supplemented in the guide.

Git helps: A designated release, complete local data, versioned examples and clear proofs of change enable independent use. The instructions for later releases are also practically tested.

▼ For workshop and self-study: a question

Where does someone outside our team need knowledge that is only in our minds or on our computers?

THEME 6 / DOWNLOADS AND ADDITIONAL SERVICES

The library is waiting for the server.

Local data is our basic offer.

Public and still not usable

Elif wants to compare many puzzles locally. The library loads them individually via the API. The server throttles the requests; their investigation stops.

An Independent Way of Work

The team offers the complete release for download and lets the library also read local files. A small sample leads to the first result. API and frontend remain additional accesses.

REQUESTS

corpus analyses must not depend on the current project server.

Our basic offer is versioned local data and documented exports. We set up an additional service only for a specific need and with secure operation.

▼ Example and Tip

Example: To try out: userstart.zip. For the complete analysis: the designated corpus package. Both describe their files and the respective scope.

Tip: Offer small entry data and complete release separately; share large collection meaningfully.

TEI + documented exports

Independently usable data; examples and instructions are necessary for the start.

▼ Example and Tip

Example: A data package can be inspected locally using different XML tools.

Tip: This is our basic offer. Examples and transformations are part of it.

Library or static view

Facilitates specific tasks; software and examples must remain up to date.

▼ Example and Tip

Example: A Python library reads both downloaded files and data via an API.

Tip: Make the local way reliable first; a surface complements the data.

Current Application / API

Additional access with operation, security, maintenance and support.

▼ Example and Tip

Example: The server provides individual data, but needs to be maintained and can limit access.

Tip: To set up an additional service only if there is a specific need and promised operation.

That's how we'd do it.

You can do that.

1. Offer small entry data and complete release separately; share large collection meaningfully.
2. Library and examples can also be executable with local data.
3. API or Frontend supplement only if benefits, responsibility and operation are clarified.

Concrete example

To try out: userstart.zip. For the complete analysis: the designated corpus package. Both describe their files and the respective scope.

Done if: The investigation can also be continued if the API fails.

Git helps: Git versions data and software. It does not finance or operate any additional services.

▼ For workshop and self-study: a question

What concrete added value does what business promise justify?

THEME 7 / PORTABILITY AND CONSERVATION

The host is moving, so I want the research to stay.

Get files and necessary context together.

Move more than files

When changing the host, the Git files are present. Linked discussions and release attachments are missing. Some decisions are only explained there.

Maintaining the needed context

Jonas takes over the necessary platform information and records professional decisions in the repo. The tests also run outside of the previous automation. Archived releases secure the earlier research references.

REQUESTS

Git history and platform stock are different things.

We secure Git, published packages and the necessary decisions for understanding. The rebuild is checked using a versioned guide.

▼ Example and Tip

Example: Git transfers commits and tags. Release attachments, relevant discussions and service settings are additionally adopted or documented in a comprehensible manner.

Tip: Back up Git references and required additional files; export or document relevant issues and decisions.

Git data

file versions, commit relationships and transferred references. Check tags and required additional objects explicitly.

Additional received

Relevant issues and decisions, attachments, test procedures, CI configuration, roles, accesses and contact paths.

Received releases, complete packages and an actual rebuild limit dependencies. The archive does not automatically replace a running host.

That's how we'd do it.

You can do that.

1. Back up Git references and required additional files; export or document relevant issues and decisions.
2. Execute checks and exports without the old CI environment.
3. Check release packages, citations, contact and permissions at the new location.

Concrete example

Git transfers commits and tags. Release attachments, relevant discussions and service settings are additionally adopted or documented in a comprehensible manner.

Done if: The files are understandable at the new location and the documented procedures executable.

Git helps: Git data are portable; scientific archiving and documented procedures limit the dependence on platform functions.

▼ For workshop and self-study: a question

What would be possible after the loss of the current host?